

OlimpOS

Documentación Técnica del Sistema

Interrelación de Archivos y Funciones

Sistema de Gestión Integral para Gimnasios

Versión 1.0.0 • Flet (Python) • 2026

1. Introducción

OlimpOS es una aplicación de escritorio construida con el framework Flet (Python) para la gestión integral de gimnasios. Permite administrar socios, personal, rutinas de entrenamiento, planes nutricionales y usuarios del sistema desde una interfaz moderna y oscura.

Este documento describe la arquitectura del proyecto, explica la responsabilidad de cada archivo y detalla cómo se interrelacionan entre sí.

2. Estructura del Proyecto

El proyecto sigue una arquitectura en capas separando configuración, estado, navegación, componentes y vistas:

main.py	← Punto de entrada
app/	
__init__.py	
config.py	← Constantes globales
router.py	← Navegación
state.py	← Estado global (datos + sesión)
components/	
__init__.py	
ui.py	← Componentes reutilizables
layout.py	← Shell (sidebar + contenido)
views/	
__init__.py	
login.py	← Pantalla de inicio de sesión
dashboard.py	← Vista principal
socios.py	← Gestión de socios
personal.py	← Gestión de empleados
rutinas.py	← Catálogo de rutinas
nutricion.py	← Planes nutricionales
usuarios.py	← Gestión de accesos (admin)

2.1 Inventario de Archivos

Archivo	Responsabilidad	Capa
main.py	Punto de entrada. Configura la ventana y arranca el login.	Entrada
config.py	Constantes: colores, rutas, menú, dimensiones, tipografía.	Config
state.py	Estado global: sesión de usuario y datos mock (socios, etc.).	Estado
router.py	Navegación: guards de auth/rol, renderizado de vistas.	Navegación

<code>components/ui.py</code>	Librería de widgets reutilizables del sistema de diseño.	Componentes
<code>components/layout.py</code>	Shell principal: sidebar fijo + área de contenido.	Componentes
<code>views/login.py</code>	Vista de login y función de carga del shell post-login.	Vista
<code>views/dashboard.py</code>	Vista del dashboard con estadísticas y actividad reciente.	Vista
<code>views/socios.py</code>	Vista de gestión de socios con tabla y búsqueda en tiempo real.	Vista
<code>views/personal.py</code>	Vista de gestión de personal en grilla de tarjetas.	Vista
<code>views/rutinas.py</code>	Vista del catálogo de rutinas con barra de progreso.	Vista
<code>views/nutricion.py</code>	Vista de planes nutricionales con estadísticas y macros.	Vista
<code>views/usuarios.py</code>	Vista exclusiva de admins para gestionar accesos al sistema.	Vista

3. Flujo de Ejecución

El sistema sigue el siguiente orden al arrancar:

- `main.py` arranca: configura la ventana, crea el router con `init_router()` y llama a `show_login()`.
- `show_login()` (`login.py`) construye el layout de dos paneles y lo agrega a la página.
- El usuario ingresa credenciales y presiona Enter o el botón. `handle_login()` llama a `app_state.login()`.
- Si el login es exitoso, `_load_main_app()` construye el shell (sidebar + Dashboard) y registra el `content_ref` en el router.
- El usuario navega haciendo clic en el sidebar. Cada clic llama a `router.navigate(ruta)`.
- El router valida guards (autenticación y rol), luego instancia la vista y reemplaza el contenido del container.
- Al hacer logout, `app_state.logout()` limpia la sesión y `show_login()` reconstruye la pantalla de login.

Patrón clave: Las vistas NUNCA reconstruyen la página completa. Solo reemplazan el `.content` del Container referenciado por `content_ref`. Esto evita parpadeos y mejora el rendimiento.

4. Capas y Responsabilidades

4.1 Capa de Configuración — `config.py`

Centraliza todas las constantes del sistema. Ningún valor está hardcodeado en los componentes: todos importan desde `config.py`.

Función / Clase	Descripción	Usada por
<code>class Colors</code>	Define la paleta completa en formato hex. Todos los componentes importan colores exclusivamente desde aquí.	<i>Todos los archivos</i>
<code>class Fonts</code>	Define los nombres de las fuentes registradas en <code>main.py</code> (Bebas Neue, DM Sans).	<i>ui.py, vistas</i>
<code>class Routes</code>	Constantes de rutas ('dashboard', 'socios', etc.). El router y los botones de navegación las usan.	<i>router.py, vistas</i>
<code>NAV_ITEMS</code>	Lista de dicts con label, icono y ruta de cada ítem del menú lateral. Usada por <code>build_sidebar()</code> .	<i>ui.py</i>
<code>SIDEBAR_WIDTH, etc.</code>	Dimensiones (ancho sidebar, alto topbar, tamaño ventana). Usadas en <code>ui.py</code> y <code>main.py</code> .	<i>main.py, ui.py</i>

4.2 Capa de Estado — state.py

Implementa el patrón Singleton. La instancia global `app_state` es la única fuente de verdad de la aplicación. Cada método marcado con TODO será reemplazado por una llamada HTTP al backend.

Función / Clase	Descripción	Usada por
<code>app_state.login()</code>	Autentica al usuario comparando con datos mock. Devuelve (bool, mensaje). TODO: POST /api/auth/login.	<i>login.py</i>
<code>app_state.logout()</code>	Limpia la sesión (logged_in=False, current_user=None). Llamado desde el botón de logout del sidebar.	<i>ui.py</i>
<code>app_state.is_admin()</code>	Retorna True si el rol del usuario activo es 'admin'. Usado por el guard de roles del router.	<i>router.py</i>
<code>app_state.get_user_name()</code>	Retorna el nombre del usuario activo para mostrarlo en la UI (topbar del dashboard, sidebar).	<i>dashboard.py, ui.py</i>
<code>app_state.get_socios()</code>	Retorna la lista de socios. TODO: GET /api/socios.	<i>socios.py, dashboard.py</i>
<code>app_state.get_personal()</code>	Retorna la lista de empleados. TODO: GET /api/personal.	<i>personal.py</i>
<code>app_state.get_rutinas()</code>	Retorna el catálogo de rutinas. TODO: GET /api/rutinas.	<i>rutinas.py</i>
<code>app_state.get_planes_nutricion()</code>	Retorna los planes nutricionales. TODO: GET /api/nutricion.	<i>nutricion.py</i>
<code>app_state.get_dashboard_stats()</code>	Retorna las 4 métricas del dashboard. TODO: GET /api/stats.	<i>dashboard.py</i>
<code>app_state.get_actividad_reciente()</code>	Retorna los últimos 5 eventos del sistema. TODO: GET /api/actividad.	<i>dashboard.py</i>

4.3 Capa de Navegación — router.py

El Router es el controlador central de navegación. Aplica guards de seguridad antes de cada transición de pantalla.

Función / Clase	Descripción	Usada por
<code>init_router(page)</code>	Crea la instancia global del Router. Llamada UNA vez desde main.py al arrancar.	<i>main.py</i>
<code>router.setup(ref)</code>	Registra el content_ref del layout. Llamado desde <code>_load_main_app()</code> tras el login.	<i>login.py</i>
<code>router.navigate(route)</code>	Punto de entrada de toda navegación. Aplica guards y luego llama a <code>_render_view()</code> .	<i>sidebar (ui.py), botones de vistas</i>
<code>router._render_view()</code>	Instancia la vista correspondiente a la ruta y reemplaza el contenido del container.	<i>Interno — llamado por navigate()</i>
<code>router._go_login()</code>	Redirige al login cuando el guard de autenticación falla.	<i>Interno</i>
<code>get_router()</code>	Retorna la instancia global del router para acceso desde cualquier módulo.	<i>Disponible globalmente</i>

4.4 Capa de Componentes — components/ui.py

Funciona como la librería de diseño del sistema. Todas las vistas consumen estos componentes para garantizar consistencia visual.

Función / Clase	Descripción	Usada por
<code>build_sidebar()</code>	Construye el panel lateral completo: logo, menú con ítem activo resaltado, footer con logout.	<i>login.py</i> (<i>_load_main_app</i>)
<code>_nav_item()</code>	Ítem individual del menú con hover animation e indicador naranja del ítem activo.	<i>Interno de build_sidebar()</i>
<code>build_topbar()</code>	Barra superior de cada sección con título, subtítulo y botones de acción.	<i>Todas las vistas</i>
<code>stat_card()</code>	Tarjeta de estadística con valor grande, tendencia (flecha) y color por categoría.	<i>dashboard.py</i>
<code>status_badge()</code>	Badge de estado tipo pill coloreado: Activo (verde), Vencido (rojo), etc.	<i>socios.py, personal.py, usuarios.py, dashboard.py</i>
<code>level_badge()</code>	Badge de nivel de dificultad: Principiante (verde), Intermedio (amarillo), Avanzado (rojo).	<i>rutinas.py</i>
<code>primary_button()</code>	Botón naranja con ícono opcional y hover. Botón principal de acción de cada vista.	<i>Todas las vistas</i>

<code>input_field()</code>	TextField estilizado: colores, border y fuente del sistema. Soporta password y prefix icon.	<i>login.py, todos los modales</i>
<code>section_card()</code>	Container con fondo de tarjeta y borde. Opcionalmente muestra un título.	<i>dashboard.py</i>
<code>show_snack()</code>	Muestra una notificación temporal en la parte inferior de la pantalla.	<i>Todas las vistas (en callbacks de guardado)</i>
<code>confirm_dialog()</code>	Diálogo modal de confirmación genérico con callbacks Cancelar/Confirmar.	<i>socios.py (eliminar)</i>
<code>open_dialog()</code>	Agrega un AlertDialog al overlay de la página y lo abre.	<i>Todas las vistas (modales)</i>
<code>close_dialog()</code>	Cierra un AlertDialog ocultándolo del overlay.	<i>Todos los modales</i>

4.5 Capa de Layout — components/layout.py

Provee la función `build_main_layout()` que arma el shell de la aplicación. En el flujo actual, el shell se construye directamente en `login.py` para mayor control de las referencias.

Función / Clase	Descripción	Usada por
<code>build_main_layout()</code>	Crea el Row principal: sidebar fijo (240px) + Container de contenido expandible. Registra el <code>content_ref</code> en el router.	<i>login.py (alternativa)</i>

5. Capa de Vistas

Todas las vistas siguen el mismo patrón de diseño:

- Son clases Python que reciben page y router en su constructor.
- Exponen un método build() que retorna un ft.Column con toda la UI.
- Obtienen datos desde app_state (nunca los almacenan ellas mismas).
- Lllaman a router.navigate() para transicionar a otras vistas.
- Usan componentes de ui.py para construir los widgets (build_topbar, status_badge, etc.).
- Los modales (AlertDialog) se abren con open_dialog() y se cierran con close_dialog().

5.1 login.py

La vista más especial del sistema. No es una clase sino funciones que reemplazan toda la página.

Función / Clase	Descripción	Usada por
<code>show_login(page, router)</code>	Limpia la página y construye el layout de dos paneles: decorativo (izq) + formulario (der). Registra handle_login en el botón y on_key en page.on_keyboard_event.	<i>main.py, router._go_login()</i>
<code>handle_login()</code>	Valida campos, llama a app_state.login() y en éxito llama a _load_main_app(). En fallo muestra el mensaje de error.	<i>Interno — botón y tecla Enter</i>
<code>_load_main_app(page, router)</code>	Construye el shell (sidebar + Dashboard inicial), registra el content_ref en el router y reemplaza toda la página.	<i>Llamado por handle_login()</i>
<code>_feature_row(icon, text)</code>	Fila decorativa con ícono naranja y texto para la lista de características del panel izquierdo.	<i>Interno de show_login()</i>

5.2 dashboard.py

Vista de bienvenida post-login con métricas clave y accesos rápidos.

Función / Clase	Descripción	Usada por
<code>DashboardView.build()</code>	Ensambla: topbar + fila de 4 stat_cards + feed de actividad + tabla de socios recientes + panel de accesos rápidos.	<i>router._render_view()</i>
<code>_activity_item(act)</code>	Widget de ítem de actividad: ícono coloreado según tipo + descripción + hora relativa.	<i>Interno — build()</i>
<code>_quick_btn(label, ...)</code>	Botón de acceso rápido con hover animation que navega a otra sección al hacer clic.	<i>Interno — build()</i>
<code>_socio_row(s)</code>	Fila de la lista de socios recientes: avatar + nombre/plan + badge de estado.	<i>Interno — build()</i>

5.3 socios.py

Vista de gestión completa de socios con búsqueda en tiempo real.

Función / Clase	Descripción	Usada por
<code>SociosView.build()</code>	Construye topbar + barra de búsqueda con chips de filtro + tabla de socios.	<code>router._render_view()</code>
<code>SociosView._build_table()</code>	Genera la tabla con encabezado fijo y una fila por cada socio de la lista. Se llama también al filtrar.	<i>Interno — <code>build()</code> y <code>_on_search()</code></i>
<code>SociosView._table_row(s)</code>	Fila de la tabla: avatar circular + nombre + plan + badge de estado + fecha + botones editar/eliminar.	<i>Interno — <code>_build_table()</code></i>
<code>SociosView._on_search(e)</code>	Callback de búsqueda: filtra la lista por nombre o plan (case-insensitive) y reconstruye la tabla en el Ref.	<code>TextField on_change</code>
<code>SociosView._open_form()</code>	Modal de creación/edición. En edición pre-carga los datos del socio en los campos via Ref.	<i>Botones de la tabla</i>
<code>SociosView._confirm_delete()</code>	Abre diálogo de confirmación antes de eliminar un socio. Usa <code>confirm_dialog()</code> genérico.	<i>Botón eliminar de la tabla</i>
<code>_filter_chip(label, active)</code>	Chip de filtro decorativo (Todos/Activos/Vencidos) con estilo activo/inactivo.	<i>Interno — <code>build()</code></i>

5.4 personal.py

Gestión del equipo de empleados en formato de grilla de tarjetas.

Función / Clase	Descripción	Usada por
<code>PersonalView.build()</code>	Topbar + ResponsiveRow de tarjetas de personal (1/2/4 columnas según ancho de ventana).	<code>router._render_view()</code>
<code>PersonalView._staff_card(p)</code>	Tarjeta de empleado: avatar + nombre + rol con ícono + chip de turno coloreado + acciones.	<i>Interno — build()</i>
<code>PersonalView._open_form()</code>	Modal crear/editar con campos de nombre, rol, turno y email. Pre-carga datos en edición.	<i>Botón 'Editar' de la card</i>

5.5 rutinas.py

Catálogo de rutinas de entrenamiento con indicador de ocupación.

Función / Clase	Descripción	Usada por
<code>RutinasView.build()</code>	Topbar + ResponsiveRow de tarjetas de rutinas.	<code>router._render_view()</code>
<code>RutinasView._rutina_card(r)</code>	Tarjeta: ícono + level_badge + nombre + pills de frecuencia/duración + ProgressBar de ocupación + botones.	<i>Interno — build()</i>
<code>RutinasView._open_form()</code>	Modal de creación con nombre, nivel, días, duración y descripción multilínea.	<i>Botón 'Nueva Rutina'</i>
<code>RutinasView._open_detail(r)</code>	Modal de solo lectura con detalles de la rutina (nivel, frecuencia, duración, asignados, descripción).	<i>Botón 'Ver detalles'</i>
<code>_info_pill(icon, text)</code>	Pequeño chip con ícono y texto para mostrar frecuencia y duración en la tarjeta.	<i>Interno — _rutina_card()</i>
<code>_detail_row(label, value)</code>	Fila label:valor usada en el modal de detalle.	<i>Interno — _open_detail()</i>

5.6 nutricion.py

Administración de planes nutricionales con estadísticas calóricas.

Función / Clase	Descripción	Usada por
<code>NutricionView.build()</code>	Topbar + fila de 4 KPIs calóricos + ResponsiveRow de tarjetas de planes.	<code>router._render_view()</code>

<code>NutricionView._plan_card(p)</code>	Tarjeta: ícono de objetivo + nombre + calorías en número grande + asignados + botón 'Ver plan'.	<i>Interno — build()</i>
<code>NutricionView._open_form()</code>	Modal de creación con nombre, objetivo (dropdown), calorías, macros (proteínas/carbos/grasas) y notas.	<i>Botón 'Nuevo Plan'</i>
<code>NutricionView._open_detail(p)</code>	Modal de detalle con objetivo, calorías y barras de progreso de distribución de macronutrientes (30/50/20%).	<i>Botón 'Ver plan'</i>
<code>_cal_stat(label, value, ...)</code>	Tarjeta KPI compacta con ícono, valor y label. Usada en el resumen calórico superior.	<i>Interno — build()</i>
<code>_macro_bar(label, pct, color)</code>	Fila con ProgressBar para visualizar la distribución porcentual de un macronutriente.	<i>Interno — _open_detail()</i>
<code>OBJETIVO_CONFIG</code>	Dict que mapea objetivo nutricional a color, fondo e ícono. Usado en tarjetas y modal de detalle.	<i>Interno</i>

5.7 usuarios.py

Vista de administración de accesos. Solo accesible para el rol admin (guard en el router).

Función / Clase	Descripción	Usada por
<code>UsuariosView.build()</code>	Topbar + banner de advertencia + tabla de usuarios del sistema + tabla de permisos por rol.	<i>router._render_view()</i>
<code>UsuariosView._user_row(u)</code>	Fila: avatar coloreado por rol + nombre/@username + badge de rol + badge de estado + 3 botones de acción.	<i>Interno — build()</i>
<code>UsuariosView._open_form()</code>	Modal crear/editar. En creación incluye campo de contraseña inicial y nota informativa (expansión condicional).	<i>Botón 'Nuevo Usuario', botón editar</i>
<code>UsuariosView._reset_password(u)</code>	Diálogo de confirmación para simular el reseteo de contraseña. TODO: POST /api/usuarios/{id}/reset-password.	<i>Botón de llave en la tabla</i>
<code>_table_header()</code>	Encabezado de la tabla de usuarios: Usuario, Rol, Estado, Acciones.	<i>Interno — build()</i>
<code>_perms_row(rol, cfg)</code>	Fila de la tabla de permisos: ícono del rol + nombre + chips de secciones accesibles.	<i>Interno — build()</i>
<code>ROLE_CONFIG</code>	Dict que mapea rol interno ('admin', 'trainer', 'staff', 'nutri') a label, color, fondo e ícono.	<i>Interno</i>

6. Mapa de Dependencias

El siguiente esquema muestra qué archivos importa cada módulo del sistema:

```
main.py          → config.py, router.py, views/login.py
router.py        → config.py, state.py, views/* (lazy imports)
state.py         → (sin dependencias internas)
config.py        → (sin dependencias internas – solo flet)
components/ui.py → config.py, state.py
components/layout.py → config.py, components/ui.py
views/login.py   → config.py, state.py, components/ui.py,
views/dashboard.py → config.py, state.py, components/ui.py
views/dashboard.py → config.py, state.py, components/ui.py
views/socios.py  → config.py, state.py, components/ui.py
views/personal.py → config.py, state.py, components/ui.py
views/rutinas.py → config.py, state.py, components/ui.py
views/nutricion.py → config.py, state.py, components/ui.py
views/usuarios.py → config.py, state.py, components/ui.py
```

Regla de dependencias: config.py y state.py son los únicos módulos sin dependencias internas. Todo el resto depende de ellos. Las vistas NUNCA importan entre sí — toda comunicación pasa por router.py y app_state.

7. Guards de Seguridad

El router implementa dos guards que se verifican en cada llamada a router.navigate():

Guard 1 — Autenticación

Si app_state.logged_in es False y la ruta destino no es 'login', el router redirige automáticamente a la pantalla de login. Esto protege todas las vistas sin necesidad de código en cada una.

Guard 2 — Roles

Si la ruta destino es 'usuarios' y app_state.is_admin() retorna False, la navegación se ignora silenciosamente. Actualmente no muestra feedback al usuario (TODO: agregar SnackBar de 'Sin permisos').

Matriz de permisos: admin: acceso total. trainer: Dashboard, Socios, Rutinas. nutri: Dashboard, Socios, Nutrición. staff: Dashboard, Socios.

8. Integración con Backend (TODO)

Todos los métodos de `app_state` que retornan datos están marcados con TODO. Para conectar con un backend real, cada método debe reemplazarse con una llamada HTTP:

```
app_state.login()           → POST /api/auth/login
app_state.get_socios()      → GET /api/socios
app_state.get_personal()    → GET /api/personal
app_state.get_rutinas()     → GET /api/rutinas
app_state.get_planes_nutricion() → GET /api/nutricion
app_state.get_dashboard_stats() → GET /api/stats
app_state.get_actividad_reciente() → GET /api/actividad?limit=5
SociosView._save_socio()    → POST/PUT /api/socios
PersonalView._save()        → POST/PUT /api/personal
RutinasView._save()        → POST /api/rutinas
NutricionView._save()      → POST /api/nutricion
UsuariosView._save()       → POST/PUT /api/usuarios
UsuariosView._reset_password() → POST /api/usuarios/{id}/reset-password
```

9. Convenciones del Código

- Los imports diferidos (dentro de funciones) en `router.py` evitan dependencias circulares en el momento de carga del módulo.
- El patrón `lambda e, x=s: ...` en los botones de filas captura el valor actual de la variable de loop, evitando el bug clásico de closure en Python.
- Todos los `ft.Ref` se usan para leer/modificar controles Flet desde callbacks definidos fuera del árbol de widgets.
- Las vistas usan `ft.ScrollMode.AUTO` en la Column principal para manejar contenido que supera la altura de la ventana.
- El `ft.ResponsiveRow` con el atributo `col={...}` permite layouts que se adaptan a distintos anchos de ventana sin código adicional.
- Los colores con transparencia se expresan como strings hex de 8 caracteres: `'#FF572240'` (40 en hex $\approx$ 25% de opacidad).
- Ninguna vista almacena datos propios — todo estado vive en `app_state`. Las vistas son puras funciones de renderizado.